

Algoritmos y Estructuras de Datos 1

2025-01-01

Contents

1	Programación Orientada a Objetos	1
1.1	Tipos de Datos	1
1.2	Tipo de dato simple	1
1.3	Tipo de dato compuesto	2
1.4	Tipo referencia y tipo valor	2
1.5	Tipos de Datos Abstractos	2
1.6	Programación Orientada a Objetos	3
1.7	Definición de Objeto	3
1.8	Definición de Clase	5
1.9	Introducción	5
1.10	Abstracción	6
1.11	Encapsulamiento	6
1.12	Herencia	8
1.13	Polimorfismo	9
1.14	Run-time (tiempo de ejecución)	9
1.15	Compile-time	11
2	Estructuras de Datos Lineales	13
2.1	Tipo de Dato Abstracto Lista	13
2.2	Implementación mediante arreglos	13
2.3	Implementación mediante listas simples enlazadas	15
2.3.1	Estructura general en Java	15
2.4	Implementación mediante lista doblemente enlazada	17
2.4.1	Estructura general en Java	17
2.4.2	Aplicabilidad	19
2.5	Implementación mediante lista enlazada circular	20
2.5.1	Estructura general en Java	20
2.5.2	Aplicabilidad	22
2.6	Tabla comparativa de implementaciones	22
2.7	Pilas	22
2.8	Tipo de dato abstracto Pila	22
2.9	Implementación de pilas con arreglos	23
2.10	Implementación de pilas con listas enlazadas	24

2.11 Colas	25
2.12 Tipo de dato abstracto Cola	25
2.13 Implementación de colas con arreglos	25
2.14 Implementación de colas con listas enlazadas	28
2.15 Colas de prioridad	29
2.16 Tipo de dato abstracto Cola de prioridad	29
2.17 Tipos de colas de prioridad	30
2.18 Implementación de colas de prioridad con listas enlazadas	30
2.19 Generics	31

Chapter 1

Programación Orientada a Objetos

En esta sección se introducen algunos conceptos esenciales sobre tipos de datos que será esencial para la comprensión de la programación orientada a objetos.

1.1 Tipos de Datos

Un *tipo de dato* es una clasificación que especifica qué tipo de valores puede tomar una variable, así como las operaciones que se pueden realizar con esos valores. Los tipos de datos se utilizan en la declaración de variables y en la definición de funciones, entre otros usos.

Por ejemplo, en `Python`, los tipos de datos más comunes son:

- **Enteros (`int`):** Números enteros como 1, 20, -5.
- **Flotantes (`float`):** Números con decimales como 3.14
- **Cadenas (`str`):** Secuencias de caracteres como “Hola, mundo!”
- **Booleanos (`bool`):** Valores de verdad como `True` o `False`

Con respecto a las operaciones que se pueden realizar con estos tipos de datos, por ejemplo, se pueden realizar operaciones aritméticas con enteros y flotantes, concatenar cadenas, y realizar operaciones lógicas con booleanos.

1.2 Tipo de dato simple

Un *tipo de dato simple* es un tipo de dato que representa un único valor. Los tipos de datos simples son los tipos de datos básicos que se utilizan para representar valores individuales. No tiene sentido práctico separarlos en partes más pequeñas.

1.3 Tipo de dato compuesto

Un *tipo de dato compuesto* es un tipo de dato que representa una colección de valores. Los tipos de datos compuestos se utilizan para representar estructuras de datos más complejas que contienen múltiples valores de otros tipos de datos. Por ejemplo, un tipo de dato Cliente, puede contener los datos de nombre, edad, dirección, etc.

1.4 Tipo referencia y tipo valor

Un *tipo de referencia* es un tipo de dato que almacena una referencia a un objeto en memoria. Los tipos de referencia se utilizan para representar objetos que pueden ser compartidos y modificados por múltiples partes de un programa. Por ejemplo, en Python, las listas y los diccionarios son tipos de referencia.

Un *tipo de valor* es un tipo de dato que almacena un valor directamente en la memoria. Los tipos de valor se utilizan para representar valores que no pueden ser compartidos ni modificados por múltiples partes de un programa. Por ejemplo, en Python, los enteros y los flotantes son tipos de valor.

“Usualmente” los tipos de datos simples son tipos de valor, mientras que los tipos de datos compuestos son tipos de referencia.

1.5 Tipos de Datos Abstractos

Un *tipo de dato abstracto (TDA)* es un modelo matemático que define un conjunto de valores y un conjunto de operaciones que se pueden realizar con esos valores. Los TDAs son una forma de abstracción que permite a los programadores trabajar con datos de manera más abstracta y genérica.

Un TDA especifica una interfaz que define las operaciones que se pueden realizar con los datos, **pero no especifica cómo se implementan esas operaciones**. Esto permite a los programadores utilizar los TDAs sin tener que preocuparse por los detalles de implementación subyacentes. Por tanto, se puede decir que un TDA tiene una vista lógica y una vista física o de implementación.

```
classDiagram
class List {
    AddFirst(element: int)
    AddLast()
    DeleteFirst()
    DeleteLast()
    Find()
    Clear()
}
<<Interface>> List
```

```
class ArrayList {  
    internalArray: int[];  
}
```

```
class LinkedList {  
    First: Node;  
}
```

```
ArrayList ..|> List  
LinkedList ..|> List
```

En el diagrama anterior, se ilustra como un TDA Lista, puede ser implementado mediante Nodos con memoria dinámica o mediante un arreglo con memoria estática. El API expuesto por el tipo Lista, no debe dar detalles de cómo se implementa internamente.

Una estructura de datos se puede entender como la implementación de TDA. En Programación Orientada a Objetos, un TDA + implementación forman una clase. Algunos lenguajes permiten definir *interfaces* que son un TDA puro.

1.6 Programación Orientada a Objetos

Paradigma de programación que modela conceptos del mundo real como *objetos* que tienen atributos y comportamientos. Los objetos son ciudadanos de primera clase en la programación orientada a objetos (POO), lo que significa que pueden ser manipulados y pasados como argumentos a funciones.

Construir software orientado a objetos implica definir clases que representan tipos de objetos y crear instancias de esas clases (objetos) para interactuar entre sí.

1.7 Definición de Objeto

Estructura de datos que agrupa atributos (datos) y métodos (funciones) que operan sobre esos datos. Los objetos son instancias de clases, que definen la estructura y comportamiento de los objetos. - La estructura o características de un objeto se define mediante sus **atributos**. - El comportamiento de un objeto se define mediante sus **métodos**.

La **interfaz** de un objeto se define por los métodos y atributos públicos (considerado como una mala practica) que son accesibles desde fuera del objeto.

La mayoría de lenguajes de programación orientados a objetos, requieren la definición de clases para crear objetos.

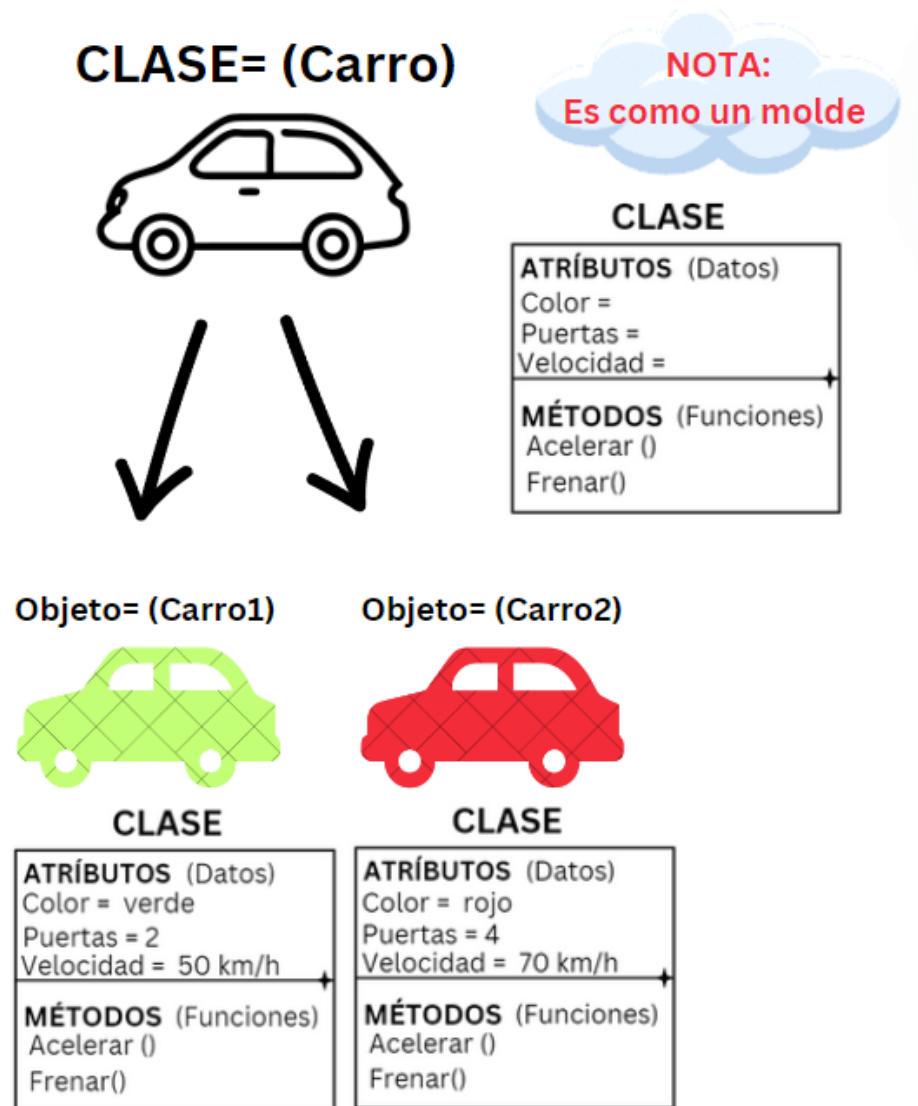


Figure 1.1: image

1.8 Definición de Clase

Las clases son *plantillas que definen la estructura y comportamiento de los objetos*. Por ejemplo, la clase Televisor en C# se puede definir de la siguiente manera:

```
class Televisor
{
    // Atributos
    string marca;
    int pulgadas;
    bool encendido;

    // Métodos
    void Encender()
    {
        encendido = true;
    }

    void Apagar()
    {
        encendido = false;
    }
}
```

Para generar objetos a partir de la clase Televisor, se utilizan las siguientes instrucciones:

```
Televisor samsung = new Televisor();
Televisor lg = new Televisor();
lg.Encender();
Console.WriteLine(samsung.encendido); // Imprime false
```

Un objeto opera sobre sus propios datos y no afecta la memoria de otros objetos diferentes. Cada objeto es un bloque de memoria independiente que contiene sus propios datos y métodos.

1.9 Introducción

Principios de POO se refiere a los conceptos fundamentales que rigen la programación orientada a objetos. Estos conceptos son la base para entender cómo se estructura y cómo se trabaja con la programación orientada a objetos. Son los elementos distintivos que diferencian la programación orientada a objetos de otros paradigmas de programación.

Los principios de POO son los siguientes: - Abstracción - Encapsulamiento - Herencia - Polimorfismo

1.10 Abstracción

Es la principal característica de POO. Permite modelar el problema por resolver en términos de objetos de alto nivel que ocultan los detalles de su implementación. La abstracción se logra a través de la creación de clases y objetos que representan entidades del mundo real. Por ejemplo, una clase `Persona` puede representar a una persona en el mundo real, con atributos como nombre, edad, etc., y métodos que permiten interactuar con la persona.

Al interactuar con objetos, pensamos en términos de su comportamiento y atributos en vez de variables y procedimientos.

La abstracción permite que los objetos se comporten como piezas de LEGO que tienen una interfaz específica para interactuar con ellos, pero no necesitamos saber cómo están contruidos internamente. En cambio, al trabajar con plastilina, no hay una clara separación, sino que todo forma parte de un todo.

1.11 Encapsulamiento

Los objetos son módulos autocontenidos que asocian código con sus datos. Los datos dentro de los objetos pueden o no exponerse según el programador lo decida. El **encapsulamiento** permite ocultar los detalles de implementación de un objeto y exponer solo la interfaz necesaria para interactuar con él.

// Example of encapsulation in C#

```
public class BankAccount
{
    private string accountNumber;
    private decimal balance;

    public BankAccount(string accountNumber)
    {
        this.accountNumber = accountNumber;
        this.balance = 0;
    }

    public decimal GetBalance()
    {
        return balance;
    }

    public void Deposit(decimal amount)
    {
        balance += amount;
    }
}
```

```
public void Withdraw(decimal amount)
{
    if (amount <= balance)
    {
        balance -= amount;
    }
    else
    {
        Console.WriteLine("Insufficient funds");
    }
}

}

public class Program
{
    public static void Main(string[] args)
    {
        BankAccount account = new BankAccount("1234567890");
        account.Deposit(1000);
        account.Withdraw(500);
        decimal balance = account.GetBalance();
        Console.WriteLine("Account balance: " + balance);
    }
}
```

Los lenguajes orientados a objetos permiten establecer el nivel de visibilidad que tiene un atributo o método con respecto a otros objetos o clases. Por ejemplo, C# soporta muchos modificadores de acceso, entre los que se incluyen:

- **public**: Accesible desde cualquier parte del código.
- **private**: Accesible solo desde la misma clase.
- **protected**: Accesible desde la misma clase y sus clases derivadas.

Por ejemplo, considera la siguiente clase Persona:

```
public class Persona
{
    private string nombre;

    public string GetNombre()
    {
        return nombre;
    }

    public void SetNombre(string nombre)
    {
        this.nombre = nombre;
    }
}
```

```
    }
}
```

En este ejemplo, el atributo nombre está declarado como `private`, lo que significa que solo es accesible desde la misma clase. Sin embargo, se proporcionan métodos públicos `GetNombre()` y `SetNombre()` para acceder y modificar el valor de nombre de manera controlada.

1.12 Herencia

- La herencia permite definir jerarquías de objetos con el objetivo de reutilizar código.
- Cada clase puede tener máximo una clase padre de la que hereda atributos y métodos.
- La clase padre se llama superclase y la clase hija se llama subclase. La superclase provee comportamiento general, mientras que las subclases proveen comportamiento especializado.
- La herencia define una relación de tipo “es un/a” entre la superclase y la subclase. Por ejemplo, si tenemos una clase `Vehículo` y una clase `Automóvil`, podemos decir que un automóvil es un vehículo.

```
// Definición de la clase base
public class Animal
{
    public string Nombre { get; set; }
    public int Edad { get; set; }

    public void Comer()
    {
        Console.WriteLine("El animal está comiendo");
    }
}

// Definición de una subclase que hereda de Animal
public class Perro : Animal
{
    public void Ladrar()
    {
        Console.WriteLine("El perro está ladrando");
    }
}

// Uso de la herencia en el programa principal
public class Program
{
    public static void Main(string[] args)
```

```

{
    // Creación de una instancia de la clase base
    Animal animal = new Animal();
    animal.Nombre = "Animal";
    animal.Edad = 5;
    animal.Comer();

    // Creación de una instancia de la subclase
    Perro perro = new Perro();
    perro.Nombre = "Firulais";
    perro.Edad = 3;
    perro.Comer();
    perro.Ladrar();
}
}

```

1.13 Polimorfismo

Etimológicamente significa “*muchas formas*” y se refiere a la capacidad de un objeto de comportarse de diferentes maneras. Hay dos tipos: *run-time* y *compile time*.

1.14 Run-time (tiempo de ejecución)

- Cuando una clase hija anula (override) un método public/protected de la clase padre. Por ejemplo:

```

public class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine("Drawing a shape");
    }
}

public class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a circle");
    }
}

public class Square : Shape
{

```

```

    public override void Draw()
    {
        Console.WriteLine("Drawing a square");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        Shape shape1 = new Circle();
        Shape shape2 = new Square();

        shape1.Draw(); // Output: Drawing a circle
        shape2.Draw(); // Output: Drawing a square
    }
}

```

- Cuando el código cliente llama el método, el método se “resuelve” en ese momento invocando el método correcto. Para encontrar el método correcto, el compilador busca en la clase real del objeto en tiempo de ejecución y en caso de no estar, sigue subiendo por la jerarquía de clases.
- Polimorfismo permite tratar la clase hija como si fuera la padre.
- Cualquier método visible de la clase Padre se puede acceder a través de la Hija.

```
Hija h= new Hija;
```

```

h. |-----| -> Aparece todo lo visible del Padre
  |         |
  |         |
  |-----|

```

- Al revés no. A través de la Padre, sólo lo que es visible de la hija que es común con la del Padre se puede acceder.

```
Padre p = new Hija;
```

```

p. |-----| -> Aparece solo lo común del padre e hija
  |         |
  |         |
  |-----|

```

- Algunos lenguajes permiten crear clases abstractas que son útiles para polimorfismo.

```

abstract class Animal{
    void respirar(){
        Console.Write("Respirando")
    }
}

```

```

    }
    abstract void(); //no tiene definición
}

class Perro : Animal {
    void comer(){
    }
}

```

- Algunos lenguajes tambien permiten crear *interfaces* las cuales proveen polimorfismo sin formar parte de una jerarquía.

```

interface Alimentable{
    void alimentar();
}

class Persona : Alimentable {
    void alimentar();
}

```

1.15 Compile-time

- La habilidad para sobrecargar (overload) métodos, es decir, crear varios métodos con el mismo nombre pero diferentes parámetros.

```

class TestClass {
    void foo() {
    }

    void foo(int x) {
    }

    void foo(string s, int x) {
    }
}

```

Abstraccion

- La abstracción es la capacidad de ignorar los detalles de partes para enfocarse en un nivel de mayor importancia. En programación orientada a objetos, la abstracción se logra a través de la creación de clases y objetos que representan entidades del mundo real. Por ejemplo, una clase **Persona** puede representar a una persona en el mundo real, con atributos como nombre, edad, etc., y métodos que permiten interactuar con la persona.

Chapter 2

Estructuras de Datos Lineales

2.1 Tipo de Dato Abstracto Lista

Pueden implementarse se muchas formas. Las más comunes son:

- **ArrayList**: Implementación mediante arreglos
- **SinglyLinkedList**: Implementación mediante listas simples enlazadas
- **DoubleLinkedList**: Implementación mediante listas doblemente enlazadas
- **CircularLinkedList**: Implementación mediante listas enlazadas circulares

2.2 Implementación mediante arreglos

```
public class ArrayList implements List {  
    private int maxSize;  
    private int currentSize;  
    private int[] storage;  
  
    public ArrayList(int size) {  
        this.maxSize = size;  
        this.currentSize = 0;  
        this.storage = new int[size];  
    }  
  
    public void clear() {  
        this.currentSize = 0;  
    }  
}
```

```

public boolean isEmpty() {
    return this.currentSize == 0;
}

public int size() {
    return this.currentSize;
}

public boolean contains(int element) {
    for (int i = 0; i < this.maxSize; i++) {
        if (this.storage[i] == element) {
            return true;
        }
    }
    return false;
}

public boolean add(int element) {
    if (this.currentSize > this.maxSize) {
        return false;
    }
    this.storage[currentSize++] = element;
    return true;
}

public int remove(int element) {
    int i;
    for (i = 0; i < this.currentSize; i++) {
        if (this.storage[i] == element) {
            indexToRemove = i;
        }
    }
    if (i == this.maxSize) {
        throw new NoSuchElementException();
    }
    for (int j = i; j < this.currentSize - 1; j++) {
        this.storage[j] = this.storage[j + 1];
    }
    this.currentSize--;
}
}

```

- Algunos lenguajes explícitamente proveen soporte para ArrayList
- **La principal ventaja** que proveen en vez de usar directamente es la abstracción de más alto nivel. El programador no tiene que lidiar con

shifts, re-sizes u otros problemas de usar arreglos directamente

- Una estrategia que se puede utilizar para evitar lanzar una excepción al llegar al tamaño máximo, es crear un nuevo array de mayor tamaño y copiar los elementos del array actual. Pero si tendrá un *hit* de performance.

2.3 Implementación mediante listas simples enlazadas

Todos los enfoques utilizando listas enlazadas utilizan memoria dinámica en vez de un arreglo, es decir, asigna memoria en el *heap* cada vez que un elemento se agrega

Dado que crea la memoria *justo en el momento*, cada elemento de la lista está separado en la memoria (a diferencia de un array donde todos los elementos están en memoria contigua)

- Cada elemento de la lista es un *nodo*. Cada nodo tiene dos partes:
 - El **valor**: es el elemento relevante para el programador. El valor que solicitó registrar en la lista. Por ejemplo, `add(3)`, el 3 es el valor de interés para el programador
 - Una **referencia** al nodo siguiente que actua como el elemento que encadena la lista.
- Visualmente, una lista enlazada se puede representar como:
 - Como se puede notar, el último elemento apunta a *null* indicando el fin de la lista
 - Es esencial llevar y mantener una referencia a la cabeza de la lista. Si la cabeza de la lista se pierde, se pierde toda la lista.
 - Opcionalmente y para mejorar la eficiencia de la inserción al final, se puede llevar una referencia a la cola de la lista. Este tipo se conoce como *DoubleEndedLinkedList*.

2.3.1 Estructura general en Java

```
// No es public, es una clase interna para no exponer la clase nodo a
// otras clases fuera del paquete
class Node {
    int value;
    Node next;

    public Node(int value) {
        this.value = value;
        this.next = null;
    }
}

public class SinglyLinkedList implements List {
```

```
private Node head;
private int size;

public SinglyLinkedList() {
    this.head = null;
    this.size = 0;
}

public void clear() {
    this.head = null;
    this.size = 0;
}

public boolean isEmpty() {
    return this.size == 0;
}

public int size() {
    return this.size;
}

public boolean contains(int element) {
    Node current = this.head;
    while (current != null) {
        if (current.value == element) {
            return true;
        }
        current = current.next;
    }
    return false;
}

public boolean add(int element) {
    Node newNode = new Node(element);
    if (this.head == null) {
        this.head = newNode;
    } else {
        Node current = this.head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    this.size++;
    return true;
}
```

```

public int remove(int element) {
    if (this.head == null) {
        throw new NoSuchElementException();
    }
    if (this.head.value == element) {
        this.head = this.head.next;
        this.size--;
        return element;
    }
    Node current = this.head;
    while (current.next != null) {
        if (current.next.value == element) {
            current.next = current.next.next;
            this.size--;
            return element;
        }
        current = current.next;
    }
    throw new NoSuchElementException();
}
}

```

2.4 Implementación mediante lista doblemente enlazada

- La lista doblemente enlazada es similar a la lista simple enlazada, pero cada nodo tiene una referencia al nodo anterior y al siguiente
- Visualmente, una lista doblemente enlazada se puede representar como:
- La ventaja sobre la lista simple enlazada es que se puede recorrer la lista en ambas direcciones. La desventaja es que cada nodo tiene que mantener una referencia adicional al nodo anterior, lo que consume más memoria e implica mayor complejidad en la implementación.

2.4.1 Estructura general en Java

```

class Node {
    int value;
    Node next;
    Node prev;

    public Node(int value) {
        this.value = value;
        this.next = null;
        this.prev = null;
    }
}

```

```
    }  
}  
  
public class DoubleLinkedList implements List {  
    private Node head;  
    private int size;  
  
    public DoubleLinkedList() {  
        this.head = null;  
        this.size = 0;  
    }  
  
    public void clear() {  
        this.head = null;  
        this.size = 0;  
    }  
  
    public boolean isEmpty() {  
        return this.size == 0;  
    }  
  
    public int size() {  
        return this.size;  
    }  
  
    public boolean contains(int element) {  
        Node current = this.head;  
        while (current != null) {  
            if (current.value == element) {  
                return true;  
            }  
            current = current.next;  
        }  
        return false;  
    }  
  
    public boolean add(int element) {  
        Node newNode = new Node(element);  
        if (this.head == null) {  
            this.head = newNode;  
        } else {  
            Node current = this.head;  
            while (current.next != null) {  
                current = current.next;  
            }  
            current.next = newNode;  
        }  
    }  
}
```

2.4. IMPLEMENTACIÓN MEDIANTE LISTA DOBLEMENTE ENLAZADA19

```
        newNode.prev = current;
    }
    this.size++;
    return true;
}

public int remove(int element) {
    if (this.head == null) {
        throw new NoSuchElementException();
    }
    if (this.head.value == element) {
        this.head = this.head.next;
        if (this.head != null) {
            this.head.prev = null;
        }
        this.size--;
        return element;
    }
    Node current = this.head;
    while (current.next != null) {
        if (current.next.value == element) {
            current.next = current.next.next;
            if (current.next != null) {
                current.next.prev = current;
            }
            this.size--;
            return element;
        }
        current = current.next;
    }
    throw new NoSuchElementException();
}
```

2.4.2 Aplicabilidad

- Cuando se necesita recorrer la lista en ambas direcciones, por ejemplo, en un editor de texto, donde se necesita recorrer el texto hacia adelante y hacia atrás o en un navegador web, donde se necesita recorrer el historial de navegación hacia adelante y hacia atrás.

2.5 Implementación mediante lista enlazada circular

- La lista enlazada circular es similar a la lista simple enlazada, pero el último nodo apunta al primer nodo
- Visualmente, una lista enlazada circular se puede representar como:
- Usualmente se implementan como circular doblemente enlazada.
- Para mejorar las inserciones, en vez de mantener la referencia a *head* se utiliza una referencia a *tail* únicamente:

2.5.1 Estructura general en Java

```
class Node {
    int value;
    Node next;

    public Node(int value) {
        this.value = value;
        this.next = null;
    }
}

public class CircularSinglyLinkedList {
    private Node tail;
    private int size;

    public CircularSinglyLinkedList() {
        this.tail = null;
        this.size = 0;
    }

    public void clear() {
        this.tail = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return this.size == 0;
    }

    public int size() {
        return this.size;
    }

    public boolean contains(int element) {
        if (this.tail == null) {
```



```

        return false;
    }
    Node current = this.tail.next;
    while (current != this.tail) {
        if (current.value == element) {
            return true;
        }
        current = current.next;
    }
    return current.value == element;
}

public boolean add(int element) {
    Node newNode = new Node(element);
    if (this.tail == null) {
        newNode.next = newNode;
        this.tail = newNode;
    } else {
        newNode.next = this.tail.next;
        this.tail.next = newNode;
        this.tail = newNode;
    }
    this.size++;
    return true;
}

public int remove(int element) {
    if (this.tail == null) {
        throw new NoSuchElementException();
    }
    Node current = this.tail.next;
    Node prev = this.tail;
    while (current != this.tail) {
        if (current.value == element) {
            prev.next = current.next;
            this.size--;
            return element;
        }
        prev = current;
        current = current.next;
    }
    if (current.value == element) {
        if (this.size == 1) {
            this.tail = null;
        } else {
            prev.next = current.next;

```

```

        if (current == this.tail) {
            this.tail = prev;
        }
    }
    this.size--;
    return element;
}
throw new NoSuchElementException();
}
}

```

2.5.2 Aplicabilidad

Cuando se necesita una lista que no tenga un final o un principio, por ejemplo, una lista de reproducción de música, donde la última canción apunta a la primera o una lista de tareas pendientes, donde la última tarea apunta a la primera.

2.6 Tabla comparativa de implementaciones

Implementación	Ventajas	Desventajas
ArrayList	- Acceso aleatorio rápido	- Inserciones y eliminaciones son costosas - Uso ineficiente de memoria (Un arreglo grande poco usado, sigue utilizando toda la memoria asignada)
LinkedList	- Inserciones y eliminaciones son rápidas - Uso más eficiente de la memoria aunque cada nodo tiene un <i>overhead</i> adicional	- Acceso aleatorio a los elementos es costoso

2.7 Pilas

Una pila es una estructura de datos lineal que sigue el principio de LIFO (del inglés Last In, First Out), es decir, el último elemento en entrar es el primero en salir.

2.8 Tipo de dato abstracto Pila

La pila tiene dos operaciones básicas:

- **Apilar (push):** añade un elemento a la pila.
- **Desapilar (pop):** elimina el último elemento apilado.
- **Tope (top):** permite ver el elemento que está en la cima de la pila sin desapilarlo.

```
public interface IStack {  
    void push(int element);  
    int pop();  
    int top();  
}
```

Al igual que en el caso de las listas, se pueden implementar pilas con arreglos o con listas enlazadas.

2.9 Implementación de pilas con arreglos

En la implementación de pilas con arreglos, se utiliza un arreglo unidimensional para almacenar los elementos de la pila.

```
public class ArrayStack implements IStack {  
    private int[] stack;  
    private int top;  
    private int size;  
  
    public ArrayStack(int size) {  
        this.size = size;  
        stack = new int[size];  
        top = -1;  
    }  
  
    public void push(int element) {  
        if (top == size - 1) {  
            System.out.println("Stack Overflow");  
        } else {  
            stack[++top] = element;  
        }  
    }  
  
    public int pop() {  
        if (top == -1) {  
            System.out.println("Stack Underflow");  
            return -1;  
        } else {  
            return stack[top--];  
        }  
    }  
  
    public int top() {  
        if (top == -1) {  
            System.out.println("Stack Underflow");  
            return -1;  
        }  
    }  
}
```

```

    } else {
        return stack[top];
    }
}
}

```

2.10 Implementación de pilas con listas enlazadas

```

public class LinkedListStack implements IStack {
    private Node top;

    public void push(int element) {
        Node newNode = new Node(element);
        newNode.next = top;
        top = newNode;
    }

    public int pop() {
        if (top == null) {
            System.out.println("Stack Underflow");
            return -1;
        } else {
            int element = top.data;
            top = top.next;
            return element;
        }
    }

    public int top() {
        if (top == null) {
            // Preguntar a los estudiantes, que es mejor, este enfoque o
            // lanzar excepciones como se vio en ejemplos pasados
            System.out.println("Stack Underflow");
            return -1;
        } else {
            return top.data;
        }
    }
}

// Otro enfoque para que la clase nodo solo pueda usarse dentro de
// la clase Stack
private class Node {
    int data;
}

```

```

        Node next;

        public Node(int data) {
            this.data = data;
        }
    }
}

```

2.11 Colas

Es una estructura de datos que sigue el principio de FIFO (del inglés First In, First Out), es decir, el primer elemento en entrar es el primero en salir. Respeta el orden de llegada de los elementos.

2.12 Tipo de dato abstracto Cola

```

public interface IQueue {
    void enqueue(int element);
    int dequeue();
    int front();
}

```

Cola vacía:

Agregando elementos 1, 2, 3, 4 y 5:

Quitando un elemento:

Agregando el elemento 6:

Quitando dos elementos:

2.13 Implementación de colas con arreglos

```

public class ArrayQueue implements IQueue {
    private int[] queue;
    private int front;
    private int rear;
    private int size;

    public ArrayQueue(int size) {
        this.size = size;
        queue = new int[size];
        front = -1;
        rear = -1;
    }
}

```

```

public void enqueue(int element) {
    if (rear == size - 1) {
        System.out.println("Queue Overflow");
    } else {
        if (front == -1) {
            front = 0;
        }
        queue[++rear] = element;
    }
}

public int dequeue() {
    if (front == -1) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        int element = queue[front];
        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front++;
        }
        return element;
    }
}

public int front() {
    if (front == -1) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        return queue[front];
    }
}
}

```

Esta implementación tiene un problema conocido como “drifting” que ocurre cuando se hacen muchas operaciones de inserción y eliminación. La cola se desplaza hacia la izquierda y se desperdicia espacio. Para solucionar este problema se puede implementar una cola circular.

```

public class CircularArrayQueue implements IQueue {
    private int[] queue;
    private int front;
    private int rear;
}

```

```
private int size;

public CircularArrayQueue(int size) {
    this.size = size;
    queue = new int[size];
    front = -1;
    rear = -1;
}

public void enqueue(int element) {
    if ((rear + 1) % size == front) {
        System.out.println("Queue Overflow");
    } else {
        if (front == -1) {
            front = 0;
        }
        rear = (rear + 1) % size;
        queue[rear] = element;
    }
}

public int dequeue() {
    if (front == -1) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        int element = queue[front];
        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front = (front + 1) % size;
        }
        return element;
    }
}

public int front() {
    if (front == -1) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        return queue[front];
    }
}
}
```

Con una cola circular, el frente y el final no necesariamente están al principio y al final del arreglo, respectivamente. En lugar de eso, el frente y el final se mueven a lo largo del arreglo. Cuando el final llega al final del arreglo, se mueve al principio del arreglo.

2.14 Implementación de colas con listas enlazadas

```
public class LinkedListQueue {
    private Node front;
    private Node rear;

    public LinkedListQueue() {
        front = null;
        rear = null;
    }

    public void enqueue(int element) {
        Node newNode = new Node(element);
        if (rear == null) {
            front = newNode;
            rear = newNode;
        } else {
            rear.next = newNode;
            rear = newNode;
        }
    }

    public int dequeue() {
        if (front == null) {
            System.out.println("Queue Underflow");
            return -1;
        } else {
            int element = front.data;
            front = front.next;
            if (front == null) {
                rear = null;
            }
            return element;
        }
    }

    public int front() {
        if (front == null) {
```



```

        System.out.println("Queue Underflow");
        return -1;
    } else {
        return front.data;
    }
}

private class Node {
    private int data;
    private Node next;

    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}
}

```

2.15 Colas de prioridad

Una cola de prioridad es una estructura de datos que almacena elementos en una cola, pero en lugar de seguir un orden de llegada, los elementos se organizan de acuerdo a su prioridad. Los elementos con mayor prioridad se desencolan antes que los elementos con menor prioridad.

2.16 Tipo de dato abstracto Cola de prioridad

```

public interface IPriorityQueue {
    void enqueue(int element, int priority);
    int dequeue();
    int front();
}

```

- **Encolar (enqueue):** Cuando un elemento se añade a la cosa, se mantiene el orden de acuerdo a su prioridad, reorganizando los elementos si es necesario.
- **Desencolar (dequeue):** Se elimina el elemento con mayor prioridad primero.
- **Tope (front):** Permite ver el elemento con mayor prioridad sin desencolarlo.

2.17 Tipos de colas de prioridad

- Orden ascendente: el elemento con menor valor numérico tiene mayor prioridad.
- Orden descendente: el elemento con mayor valor numérico tiene mayor prioridad.

2.18 Implementación de colas de prioridad con listas enlazadas

Las colas de prioridad se pueden implementar de muchas maneras:

- Con arreglos
- Con listas enlazadas
- Con montículos (heaps)

Más adelante en el curso, veremos la implementación con heap. Por ahora, vamos a ver una implementación con listas enlazadas.

```
public class LinkedListPriorityQueue implements IPriorityQueue {
    private Node front;
    private Node rear;

    public LinkedListPriorityQueue() {
        front = null;
        rear = null;
    }

    public void enqueue(int element, int priority) {
        Node newNode = new Node(element, priority);
        if (front == null) {
            front = newNode;
            rear = newNode;
        } else if (priority < front.priority) {
            newNode.next = front;
            front = newNode;
        } else {
            Node current = front;
            while (current.next != null && current.next.priority <= priority) {
                current = current.next;
            }
            newNode.next = current.next;
            current.next = newNode;
            if (newNode.next == null) {
                rear = newNode;
            }
        }
    }
}
```

```

    }
}

public int dequeue() {
    if (front == null) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        int element = front.element;
        front = front.next;
        return element;
    }
}

public int front() {
    if (front == null) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        return front.element;
    }
}
}

```

2.19 Generics

Hasta ahora, los ejemplos vistos de las estructuras de datos, han sido únicamente para el tipo de dato `integer`. ¿Qué pasa si queremos implementar una estructura de datos para otro tipo de dato? Por ejemplo, si queremos implementar una pila para `String` o para `double`. En este caso, tendríamos que implementar una pila para cada tipo de dato que necesitemos. Esto no es eficiente y no es escalable.

Otra opción es utilizar el tipo de dato `Object` que es la superclase de todos los tipos de datos en Java. Sin embargo, esto no es una solución óptima ya que se pierde el tipo de dato específico y se tendría que hacer un *casting* cada vez que se quiera utilizar el dato. Esto puede resultar en errores en tiempo de ejecución.

Java y muchos otros lenguajes, proveen el concepto de **generics** para solucionar este problema. Los **generics** permiten definir clases, interfaces y métodos con un tipo de dato que se especifica en el momento de la creación de la instancia.

```

public class LinkedList<T> {
    private Node<T> head;

    private static class Node<T> {
        private T data;
    }
}

```

```

        private Node<T> next;

        public Node(T data) {
            this.data = data;
            this.next = null;
        }
    }

    public void add(T data) {
        Node<T> newNode = new Node<>(data);
        if (head == null) {
            head = newNode;
        } else {
            Node<T> current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
    }

    // Other methods for LinkedList implementation...
}

/// Ejemplo de instanciación

public static void main(String[] args) {
    LinkedList<String> list = new LinkedList<>();
    list.add("Hello");
    list.add("World");
    list.add("Java");

    LinkedList<Integer> numbers = new LinkedList<>();
    numbers.add(1);
    numbers.add(2);

    LinkedList<Double> decimals = new LinkedList<>();
    decimals.add(3.14);
    decimals.add(2.71);
}

```

Una consideración importante al utilizar generics es al comparar los elementos en la estructura. Se puede aprovechar la interfaz `Comparable` de Java para forzar que los elementos de tipo `T` sean comparables entre sí.

```

public class LinkedList<T extends Comparable<T>> {
    // ...

```

```
private class Node<T extends Comparable<T>> {  
    public int compareTo(T other) {  
        return this.data.compareTo(other);  
    }  
}
```

Para cualquier tipo de datos *personalizado*, se debe implementar la interfaz **Comparable** y definir las reglas de comparación que tengan sentido dentro del contexto de la aplicación.

- Clase **Persona**: comparar por cédula
- Clase **Estudiante**: comparar por número de carné

